TUWAIQ ACADEMY
AI FUNDAMENTALS USING META TOOLS BOOTCAMP

Mirsad

مرصاد

Project Report

A proxy that routes each AI agent task to the right model and stops failure loops before any tokens are spent.

PREPARED BY

Raseel Al-zayed

Shaima Alrewaished

Faisal Alanazi

October 22, 2026

1. Problem

As AI agents move into real products, their running cost becomes a real concern, and much of it is wasted in two ways. First, an agent usually runs on a single model, chosen by the developer to handle the hardest task it might face. As a result, a simple question such as a short translation is answered by the same large, expensive model as a complex analysis, and costs the same. Second, agents can get stuck in failure loops. When a tool fails, the agent often retries the exact same call, even though the outcome cannot change. Every retry is another model call that resends the whole conversation, so the cost grows with each attempt while the task makes no progress and the user is left without an answer.

2. Solution

Mirsad (مرصاد) is a proxy that sits between an AI agent and its model provider, so every model call passes through it before any tokens are spent. To stop simple tasks from paying the price of complex ones, a small local model (Qwen 3 4B, running on Ollama) reads each user message and labels it as simple or complex. Mirsad then sends simple tasks to a smaller, cheaper model (GPT-OSS 20B) and complex tasks to a stronger one (GPT-OSS 120B), both served through Groq. To stop failure loops, Mirsad tracks every failed tool call; once the same call fails three times with the same input, it blocks the next request before it reaches the model, so the loop ends without spending more tokens. Finally, Mirsad records every call, including its tokens, latency, cost and errors, and shows them on an admin dashboard, so the team can see where money goes and catch problems early.

3. Data Collection

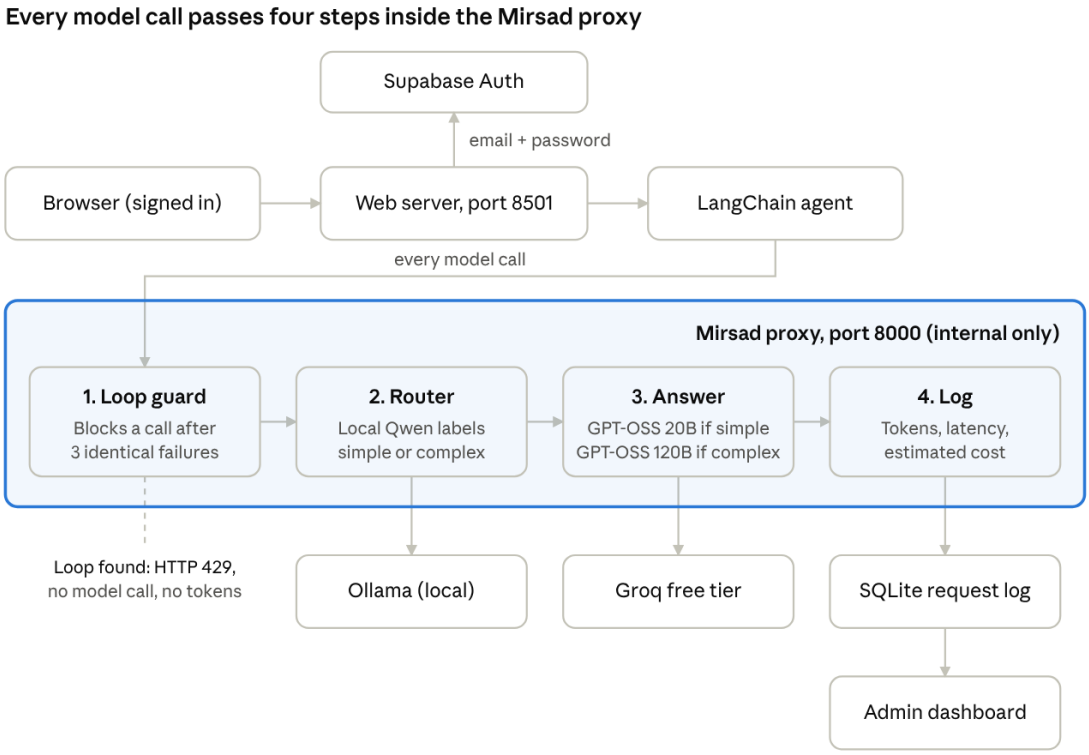
Mirsad collects its own data while it runs; no external dataset is used. Every event is stored in a local SQLite database across four tables. The `turns` table records one row for each user message, with its outcome (completed, failed, blocked or step limit) and how long the user waited for an answer. The `requests` table records one row for each model call, including the route chosen (simple or complex), the model used, input and output tokens, latency, estimated cost and status (success, error or blocked). The `provider_attempts` table records each attempt to reach Groq, including retries, with the reason for any failure, which separates a rate limit from an invalid API key. The `message_feedback` table stores users' ratings of answers (helpful or not helpful, with an optional reason). Token counts come only from what Groq reports, each row is linked to its verified user, and live traffic is kept apart from mock test traffic, so the numbers on the dashboard reflect real usage.

4. Benchmarks

Since no public benchmark covers both model routing and loop blocking for this kind of agent, we built small, purpose-made benchmarks that run offline at no cost. For routing, we wrote 100 English prompts and labelled each one by hand as simple or complex: 60 form a development set used while designing the rules, and 40 form a held-out test set that was never used for tuning, so its score shows how the router performs on prompts it has not seen. For the loop guard, we created 10 true loop scenarios, where the same failing tool call repeats 5 to 10 times, and 6 negative controls, such as a tool that fails twice and then succeeds, which must not be blocked. Finally, before any model can replace the small model, it must be cheaper than the big model on both input and output tokens and pass a bounded live comparison of at most 12 agent runs.

5. Technical Design

Mirsad runs as two services. The web server (port 8501) handles sign-in, the chat and the admin dashboard. The proxy (port 8000) is internal only: it accepts calls from the web server alone, through a shared secret token, so a browser can never reach it directly. When a user sends a message, the web server passes it to a LangChain agent with four tools (`list_files`, `read_file`, `calculator`, and `fetch_report`, which always fails so loops can be demonstrated). Every time the agent needs the model, its call goes through the proxy.

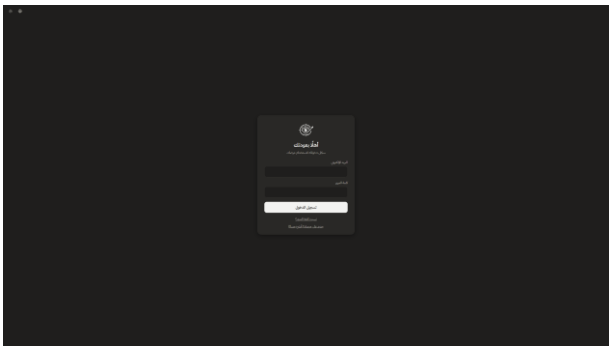


System architecture · 4 proxy steps

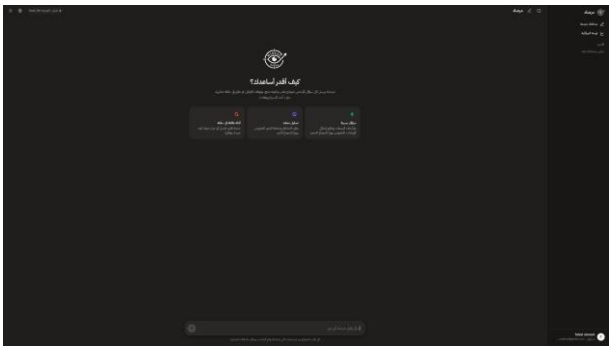
Inside the proxy, each call passes four steps in order. First, the loop guard turns every failed tool call into a fingerprint (a hash of the tool name and its arguments); if one fingerprint has failed three times, the call is rejected with HTTP 429 and no model is called. Second, the router asks the local Qwen classifier to label the user's message as simple or complex. This happens once per message, and later tool steps reuse the decision; if Ollama is down or takes longer than 20 seconds, Mirsad falls back to keyword rules and records why. Third, the chosen model answers through LiteLLM and Groq. If Groq rejects an API key, Mirsad moves to a backup key; if Groq hits its rate limit, Mirsad does not switch keys, because the limit applies to the whole account, and waits up to 10 seconds instead. Fourth, the call is logged with its tokens, latency and estimated cost.

Users sign up and sign in with email and password through Supabase Auth, so Mirsad stores no passwords. There are two roles: regular users see only the chat, while admins also see the monitoring dashboard. The admin role can only be granted from the server, never from the browser.

The screens below show the regular user's side of Mirsad. After signing in, the user lands on the chat, which offers three demo scenarios: a simple question that should go to the small model, a complex analysis that should go to the big model, and a tool stuck in a loop that Mirsad should stop. Each answer can be rated as helpful or not helpful.



Sign in



Chat home with three demo scenarios



Chat answer with helpful / not helpful buttons; past chats appear in the sidebar

6. Selected Metrics

The admin dashboard shows only metrics tied to Mirsad's actual behaviour: routing each task to the right model, stopping loops, and keeping answers working, fast and cheap. Each metric was chosen because it reveals a specific problem.

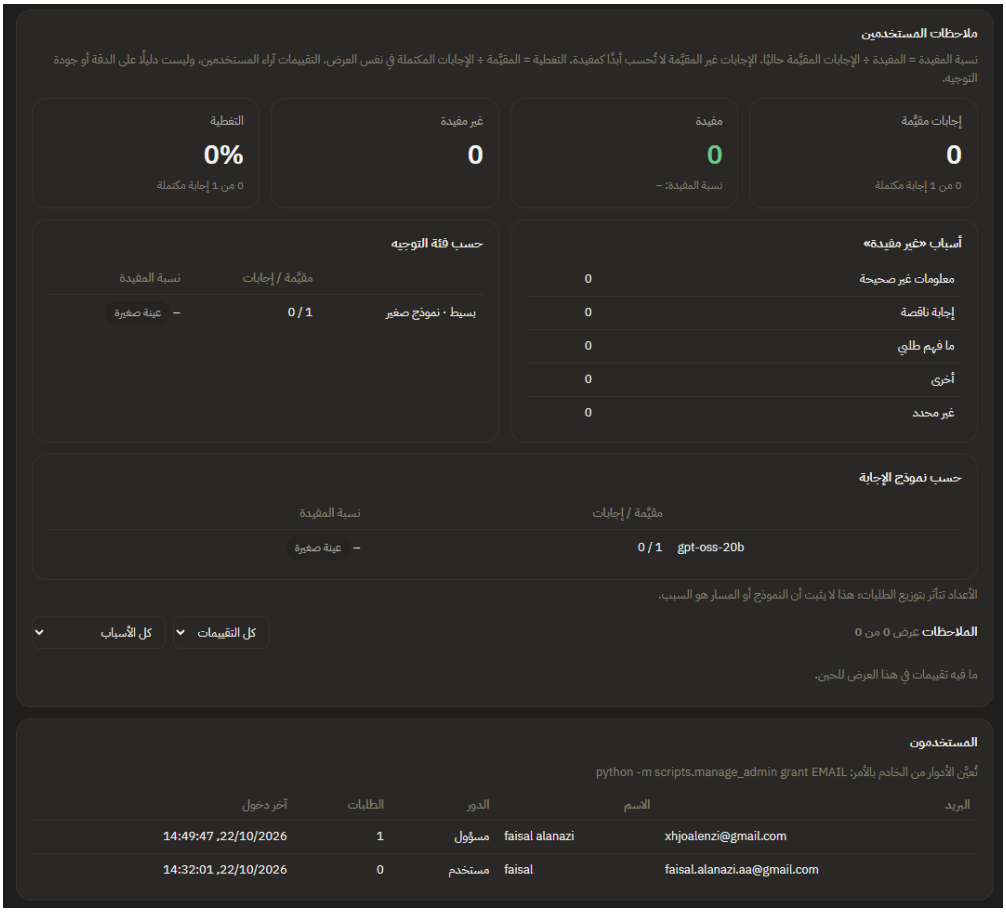


Monitoring dashboard · headline cards, route and status, cost against the big-model baseline, latency and tokens

Application health. The error rate is the share of live model calls that failed in the last five minutes. A failed call means a user did not get an answer, so a red banner appears when the rate passes 20%. Provider attempts are shown by outcome, which separates a Groq rate limit (HTTP 429) from an invalid key (HTTP 401); the two look the same to the user but need different fixes. Blocked loops are counted separately rather than as errors, and any block in the last five minutes also raises the banner, because it means an agent is stuck on a broken tool.

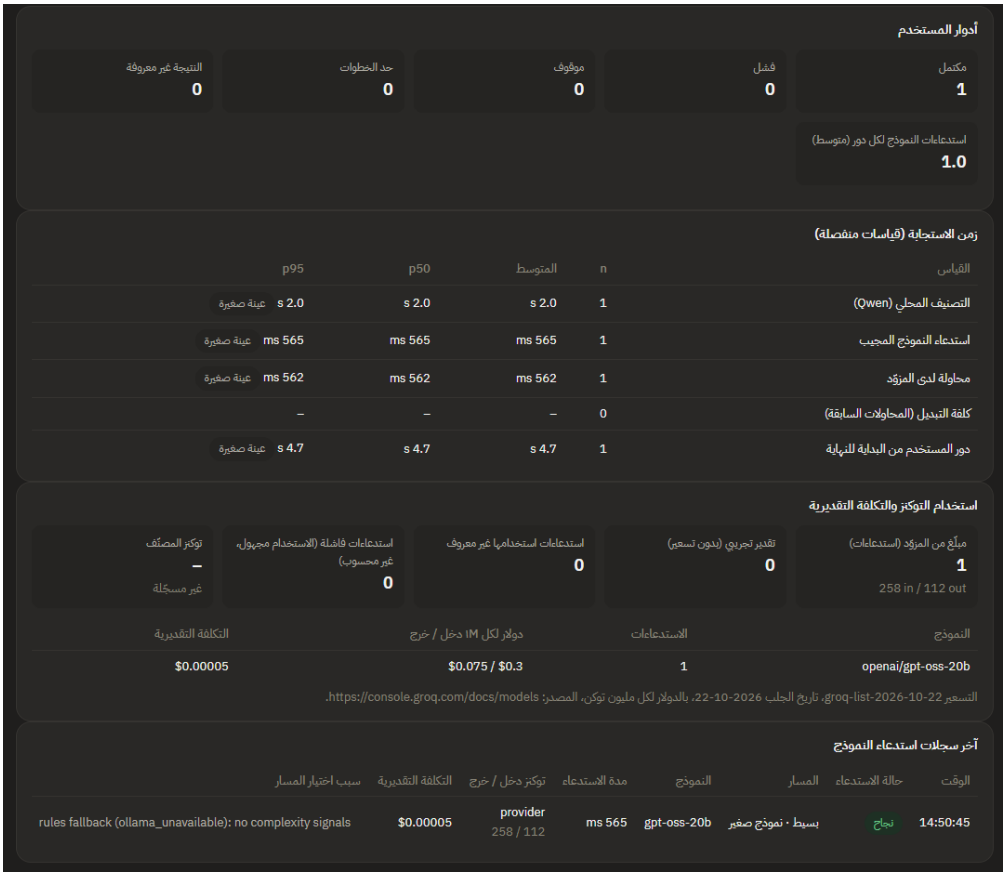
Performance. Latency is split into four parts: local classification, the model call, each provider attempt, and the full turn as the user experienced it. A single total would only say that Mirsad is slow; the split shows where the time goes, for example a slow classifier (normally 1 to 3 seconds) versus a slow provider.

AI quality and task success. The route mix shows how many messages went to the small and big models, together with the classifier's status, so a silent fallback to keyword rules is visible. Turn outcomes show whether users actually received an answer, or whether the agent failed, was blocked, or hit its step limit. Users can rate each answer as helpful or not helpful; the dashboard reports the helpful rate, how many answers were rated, and the results by route and model. This is the main signal of whether the small model answers well enough on the tasks routed to it.



Dashboard · user feedback by route and model, reasons for not helpful, and user roles

Cost. Tokens and estimated cost are shown per model, using Groq's list prices. The same tokens are also priced as if every call had gone to the big model, which shows how much routing saves and reveals when too many tasks are being sent to the big model. These are estimates at list price, not invoices, since Groq's free tier is not billed.



Dashboard · turn outcomes, split latency, token usage, estimated cost and the latest model calls

Data drift is not tracked, because Mirsad does not train a model and so has no training data to drift from. The feedback results by route and model play this role for routing quality.

Example of investigating a problem. The dashboard shows a red banner because more than 20% of calls failed in the last five minutes. The admin checks the provider attempts and sees that the failures are HTTP 429 (rate limited). This points to Groq's free-tier limit of about 30 requests per minute for the whole account, not to a broken key, so switching keys would not help; the fix is to reduce traffic or wait. If the attempts had shown HTTP 401 instead, Mirsad would already have moved to a backup key, and the admin would replace the revoked one.

7. Evaluation Results

All results below were produced offline on 22 October 2026, on the latest version of the code (commit `52b7d48`).

The keyword routing rules classified 37 of the 40 held-out prompts correctly (92.5%), and 58 of the 60 development prompts (96.7%). The held-out score is the one we report, because those prompts were never used to tune the rules. The few errors show the limits of keywords: a simple request that contains a word such as "compare" can be sent to the big model, and a complex question with no warning words can be sent to the small one. This is the reason the team added the Qwen classifier, which judges the meaning of the request rather than its words.

The loop guard stopped all 10 true loop scenarios (100%) and blocked none of the 6 healthy retry patterns (0% false alarms). Across these scenarios, it prevented 41 of 71 model calls (58%) that would otherwise have been spent on retries that could not succeed.

All 180 automated tests pass. They cover routing, the loop guard, provider failover, authentication, feedback, metrics, model selection, the proxy and the web interface, and they run offline against a temporary database, so they cost nothing.

Two things are not yet measured. The Qwen classifier has only a preliminary evaluation on examples written with AI help, so we do not claim an accuracy figure for it. Real cost savings depend on live use of the agent, which had not been recorded at the time of this report.

8. Tools

Every tool was chosen to fit two constraints: a budget of zero and two days to build.

Answering models: GPT-OSS 20B and GPT-OSS 120B on Groq. Groq's free tier gives fast answers with tool calling at no cost. The small model is listed at half the price of the big one for both input and output tokens. The team first used Qwen 3.8 27B as the small model, but found it cost more than the big model, so the rule became that price decides, not model size.

Routing classifier: Qwen 3 4B on Ollama. It runs locally for free and understands both Arabic and English. The team first planned to use RouteLLM's BERT router, but chose a local language model because incoming requests vary too much for a fixed router.

Agent: LangChain and LangGraph. They gave the team a real working agent with tools in little time, instead of a simulated one.

Model gateway: LiteLLM. It gives one interface to Groq and reports token usage, so changing a model only means changing a setting. It is pinned to version 1.106.0, because versions 1.82.7 and 1.82.8 were compromised on PyPI in March 2026.

Web server and proxy: FastAPI and Uvicorn. They are lightweight and make it easy to offer the same API format that agents already use.

Accounts: Supabase Auth. It provides free, secure sign-in, so Mirsad never stores passwords.

Storage and metrics: SQLite and pandas. SQLite needs no database server, and pandas computes the dashboard numbers from its tables.

Testing: pytest. It runs the 180 offline tests.

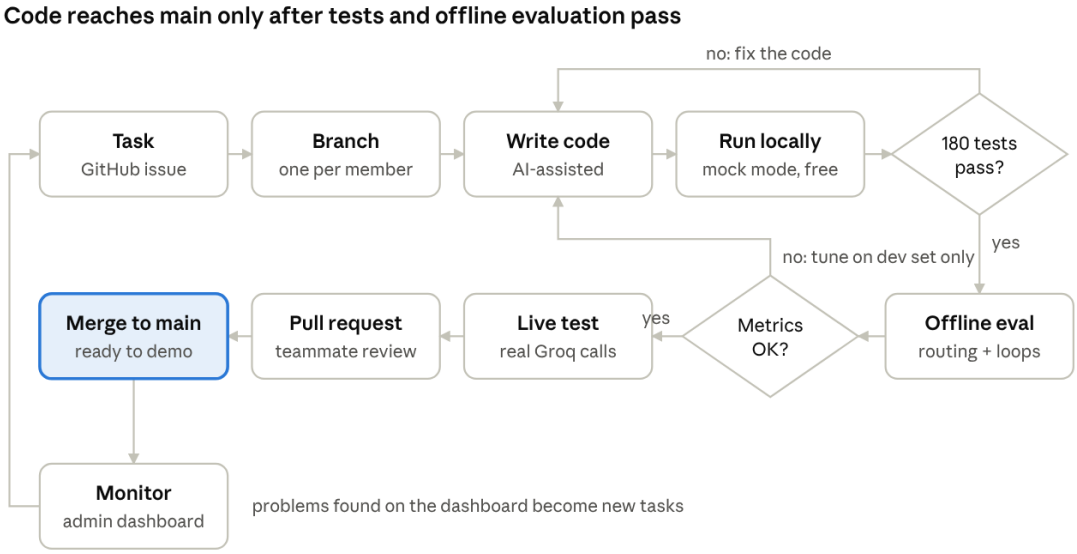
Two tools were dropped along the way. Streamlit was replaced by a custom web interface because it could not handle user sign-in, and Meta Llama models were not needed for this project.

AI assistance. Code was written with AI assistance and reviewed by a teammate before merging. This report was drafted with Claude from the repository, and every number in it was checked by running the tests and the evaluation. To verify AI-written code, the team relied on the 180 tests, tuned routing rules only on the development set, and checked prices against Groq's official model pages. [Team to confirm: which AI coding tools were used, and which parts were generated, assisted, or written by hand.]

9. Diagrams

Development Flow

The team works from one GitHub repository, with a branch for each member. All settings, thresholds and prices live in one file (`config.py`), secrets stay in a local `.env` file that is never committed, and package versions are pinned so everyone runs the same code. A mock mode (`MIRSAD_MOCK=1`) lets the whole system run without an API key: answers are simulated, while routing, the loop guard and logging stay real.



Development flow · two quality gates

A change reaches the main branch only after passing two gates. First, the 180 offline tests must pass; if they fail, the code is fixed. Second, the offline evaluation must show that routing accuracy and the loop guard still meet their targets; if not, the rules are tuned on the development set only, so the held-out score stays honest. The change is then tested live with real Groq calls, reviewed by a teammate in a pull request, and merged. Problems seen on the dashboard become new tasks, which starts the cycle again.

User Journeys

The two journeys on the next pages follow Mirsad from each persona's side, showing what the person does at each stage, what Mirsad does unseen, which data is recorded or read, and which risk each stage answers. The first follows a regular chat user from sign-in to a rated answer, including the failure path when a tool keeps failing. The second follows the team admin from an alert on the dashboard to a fix and a review of cost and quality. Each problem the admin finds becomes a new task, which feeds back into the development flow above.

USER JOURNEY · 1 OF 2

Mirsad (مرصاد): the chat user

From sign-in to a rated answer: what the user sees, and what Mirsad does on the other side of the proxy to route the task and stop failure loops.

PERSONA

Regular user

Uses the AI agent through the chat. Wants a fast, working answer and never sees the proxy, the models or the dashboard.

	STAGE 1 Sign in	STAGE 2 Ask	STAGE 3 Route	STAGE 4 Work	STAGE 5 Answer	STAGE 6 Rate
USER DOES	Signs up or signs in with email and password on the web app.	Types a question in the chat, from a short translation to a file analysis.	Waits a moment. Nothing to choose: the model is picked for them.	Watches the agent read files, run the calculator and think.	Reads the answer in the chat.	Marks the answer helpful or not helpful, with an optional reason.
MIRSAD DOES (UNSEEN)	Supabase Auth checks the credentials. A regular user gets the chat only.	The web server hands the message to the LangChain agent. Every model call goes to the internal proxy with a shared secret.	Loop guard checks failed fingerprints. Qwen 3 4B labels the message simple or complex, once. Simple goes to GPT-OSS 20B, complex to 120B.	The agent calls list_files, read_file or calculator. Later steps reuse the routing decision.	Groq answers through LiteLLM. Invalid key: backup key. Rate limit: wait up to 10 seconds.	Saves the rating with the route and model that produced the answer.
DATA RECORDED	Verified user id, linked to every later row	**turns**: one row per user message	**requests**: route chosen, status	**requests**: tokens, latency, cost	**provider_attempts** and turn outcome with wait time	**message_feedback**: rating and reason
RISK AND MIRSAD'S ANSWER	**Passwords at risk** No passwords stored by Mirsad: sign-in is Supabase's job.	**Browser bypasses the proxy** The proxy accepts the web server only, so no browser can reach it.	**Simple task, big-model price** Simple tasks go to the 20B model, listed at half the price of the 120B.	**Tool fails, agent retries forever** Three identical failures and the next call is blocked. See the failure path below.	**Slow or failed answer** Backup key on 401, bounded wait on 429, outcome always recorded.	**Weak small-model answers unnoticed** Helpful rate is reported by route and model.

FAILURE PATH **When a tool keeps failing (for example fetch_report)**

1. The tool fails
The agent retries the same call with the same input. Each retry resends the whole conversation to the model.

2. Third identical failure
Mirsad hashes the tool name and arguments into a fingerprint and counts it. The count reaches three.

3. Mirsad blocks the call
The next request is rejected with HTTP 429 before any model is called. No tokens are spent.

4. The loop ends
The turn is recorded as blocked and the user is no longer left waiting. The admin dashboard raises its red banner.

User journey 1 of 2 · the chat user

USER JOURNEY · 2 OF 2

Mirsad (مرصاد): the admin

From an alert on the dashboard to a fix: how the admin finds out where money and time go, and what is going wrong, before users complain.

PERSONA

Team admin

Runs Mirsad for the team. Sees the chat plus the monitoring dashboard, and wants to catch problems early and keep cost down.

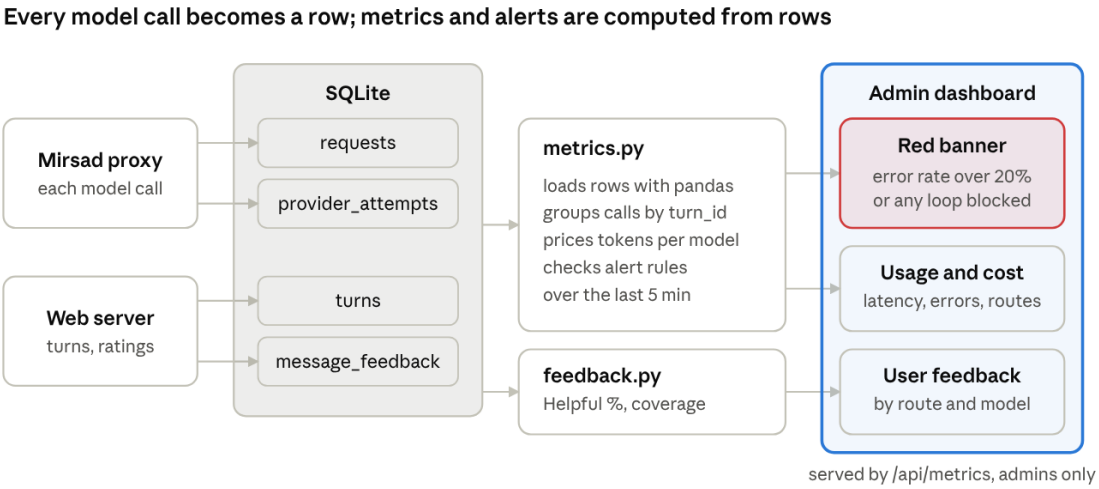
	STAGE 1 Sign in	STAGE 2 Open dashboard	STAGE 3 Spot the alert	STAGE 4 Diagnose	STAGE 5 Fix	STAGE 6 Review cost and quality
ADMIN DOES	Signs in with the admin account.	Opens the monitoring dashboard next to the chat.	Sees the red banner: more than 20% of calls failed in the last five minutes, or a loop was blocked.	Opens provider attempts by outcome: HTTP 429 (rate limit) or HTTP 401 (invalid key).	429: reduces traffic or waits. 401: replaces the revoked key. Blocked loop: fixes the broken tool.	Checks route mix, helpful rate by route and model, and cost against the all-big-model price.
MIRSAD DOES (UNSEEN)	Supabase Auth verifies the user. The admin role can only be granted from the server, never from the browser.	metrics.py and feedback.py read the four tables with pandas. Results go to admins only.	Alert rules run over the last five minutes. Blocked loops are counted apart from errors.	Shows the reason for every failed attempt, and splits latency into classifier, model call, provider attempt and full turn.	On 401 it has already moved to the backup key. On 429 it does not switch keys, because the limit covers the whole account.	Prices tokens per model at Groq list price, and again as if every call had gone to the big model.
DATA READ	Verified user and role	All four tables, live traffic only (mock excluded)	**requests** status: error or blocked	**provider_attempts**: outcome and reason	**turns**: did users get an answer again?	**requests** and **message_feedback**
WHAT IT TELLS THE TEAM	**Access is controlled** Regular users never see the dashboard.	**One view** Health, speed, quality and cost in one place.	**Early warning** Problems show up before users complain.	**Same symptom, different cause** A 429 and a 401 look alike to the user but need different fixes.	**Right fix first time** No pointless key switch when the account is rate limited.	**Is routing paying off?** Savings are visible, and so is any drop in helpful rate on the small model.

THE LOOP CONTINUES

Each problem found on the dashboard becomes a new task for the team, which restarts the development cycle. Cost figures are estimates at list price, not invoices, since Groq's free tier is not billed.

User journey 2 of 2 · the admin

Monitoring Flow



Monitoring flow · events to rows to metrics to dashboard

The proxy writes a row to `requests` and `provider_attempts` for every model call, and the web server writes a row to `turns` for every user message and to `message_feedback` for every rating. When an admin opens the dashboard, the web server reads these tables through `metrics.py` and `feedback.py`, which use pandas to group model calls by user message, price the tokens for each model, compute the metrics, and check the alert rules over the last five minutes. If the error rate is above 20% or a loop was blocked, the dashboard shows the red banner. The results are served only to admins.

10. References

1. Mirsad source code: github.com/shaimasaud5/Mirsad, commit `52b7d48`.
2. Groq, supported models and list prices, retrieved 22 October 2026: console.groq.com/docs/models.
3. Groq model pages: GPT-OSS 120B, GPT-OSS 20B, Qwen 3.8 27B.
4. Ollama, used to run the local Qwen classifier.
5. Frameworks and libraries, with versions pinned in `requirements.txt`: LangChain 1.4.6, LangGraph 1.2.16, LiteLLM 1.106.0, FastAPI 0.154.0, pandas 3.0.6, pytest 9.1.1.
6. Evaluation data in the repository: `eval/dev_prompts.csv` (60 prompts), `eval/test_prompts.csv` (40 prompts), and the loop scenarios in `eval/run_eval.py`.
7. Tuwaiq Academy, *Project 1 Instructions: AI Product Development and Monitoring*.